

产品需求说明书 / 技术方案 / 实施计划

Codex 智能任务编排平台

面向前端、后端与硬件多角色协作的自动任务执行系统

文档定位 本说明书作为产品立项、技术评审、MVP 开发、试点验收和生产化建设的统一需求基线。

文档版本	v1.0 (评审稿)
编制日期	2026 年 8 月 7 日
适用范围	公司大项目及其 frontend、backend、hardware、contracts 子项目
目标读者	产品负责人、技术负责人、平台研发、安全、DevOps、前后端与硬件团队
方案状态	需求与总体架构已形成；关键兼容性需在阶段 0 技术验证中确认

内部评审资料

文档控制

项目	内容
产品名称	Codex 智能任务编排平台
文档类型	产品需求说明书（PRD）+ 总体技术架构说明 + 实施与验收计划
当前版本	v1.0 评审稿
基线日期	2026-08-07
核心假设	公司具备 GitLab、统一身份认证、任务服务器和对象存储，开发者电脑可运行本地 Runner
变更原则	影响权限、任务状态机、GitLab 交付链路和 Codex 执行边界的变更须重新评审

关键决策摘要

- 已确认** 源码、分支、提交、Merge Request 和 CI 以 GitLab 为唯一事实来源；不允许 Codex 直接操作共享服务器源码目录。
- 已确认** 身份、角色、任务顺序、状态机、租约、审批和审计以公司任务服务器为唯一事实来源。
- 已确认** 截图、PDF、日志、视频和大型构建产物保存到 MinIO/S3 等对象存储；任务引用固定附件版本。
- 已确认** Runner 使用 OpenAI 官方 Codex SDK 或 App Server，不重新实现 Codex 的模型、聊天、代码理解、终端和审查能力。
- 已确认** 登录为前端身份时，Codex 默认可读取 frontend、backend、hardware、contracts 和授权附件，但只可写 frontend 任务工作区。
- 已确认** 每个任务使用独立 Codex 执行线程、独立任务分支和独立工作区；自动审查使用与执行隔离的上下文。
- 已确认** MVP 自动执行默认严格串行：当前任务达到业务完成条件后才领取下一项；进入 NEEDS_ATTENTION 时默认暂停队列，须人工决定重试、跳过或终止。
- 已确认** 插件自定义 UI 是首选 Codex 内入口，但不把“永久注册原生左侧一级导航项”作为已确认扩展能力；MVP 必须同时具备独立任务中心入口。
- 已确认** 生产部署、保护分支合并、破坏性数据库操作、固件烧录等高风险动作必须人工审批。

文档目录

目录将在 Microsoft Word 打开时自动更新；如未更新，请按 Ctrl+A 后按 F9。

章节导览

- 产品背景、目标、范围与成功标准
- 用户角色、身份、权限与典型业务场景
- 四列任务看板、任务状态机和详细功能需求
- 系统总体架构、Runner、Codex、GitLab 和附件方案
- 数据模型、接口设计、可靠性、安全与审计
- 部署拓扑、非功能要求、监控告警与运维策略
- 技术验证、MVP、Pilot 和 Production 实施计划
- 测试策略、验收标准、风险与待决策事项
- 附录：状态字典、示例任务、沙箱配置和参考资料

阅读建议 管理层可优先阅读产品目标、关键决策、总体架构、实施路线和风险；研发团队应完整阅读功能需求、状态机、数据模型、接口与安全章节。

产品概述

本产品是在 OpenAI 官方 Codex 能力之上构建的一套企业级任务编排、自动执行、审查审批和跨团队协作平台。平台不替代 Codex，也不重新实现代码理解、文件修改、命令执行、测试或代码审查能力；平台负责把公司任务、身份权限、GitLab 代码、附件和 Codex 执行过程组织成可持续运行、可恢复和可审计的闭环。

产品的核心用户体验是一套四列任务中心：待办、执行、审查、完成。用户可以从 Codex 任务中心、Web 或手机 App 创建多条任务；本地 Runner 自动领取当前角色队列中的第一条可执行任务，调用官方 Codex 完成修改，经过自动测试和上下文隔离的自动审查后交付 GitLab Merge Request，并按项目的队列推进策略处理下一条任务。

核心价值 把“逐条手动和 Codex 对话”升级为“由公司任务服务器驱动、按角色受控、连续执行并自动审查的研发流水线”。

业务背景与问题陈述

公司大项目通常由前端、后端、硬件/固件、接口协议和公共资料等多个相互依赖的子项目组成。不同角色需要理解全局上下文，但只应修改自己负责的代码范围。例如前端工程师需要读取后端接口实现和硬件通信协议，却不应直接获得后端或硬件仓库的写权限。

智能设备项目	
└ frontend	Web / 桌面 / 移动端前端
└ backend	服务端、数据库、接口
└ hardware	固件、嵌入式和设备协议
└ contracts	OpenAPI、Schema、通信协议
└ shared	公共文档与任务附件

- 任务需要人工逐条发送给 Codex，一个任务结束后还要手动新建下一次对话。
- 手机端产生的临时需求、截图和日志不能直接进入可自动执行的 Codex 队列。
- 任务、Codex 线程、Git 分支、测试、审查、CI 和 MR 缺少统一关联。
- Codex 缺少稳定的角色、目录和仓库权限边界，存在越权修改风险。
- 前端、后端、硬件任务和跨团队依赖缺少标准化流转方式。
- Runner 掉线、网络中断、Codex 超时或 GitLab 不可用时，任务难以安全恢复。
- 高风险操作缺少审批、审计、费用限制和全局停止能力。

产品目标

1. 允许用户一次创建任意数量的任务，并由服务器维护可靠顺序。

- 2. Runner 自动领取当前项目、当前角色、无未完成依赖的第一条任务。
- 3. 每个任务创建独立 Codex 执行线程、独立 Git 任务分支和隔离工作区。
- 4. 执行完成后自动运行测试、权限检查和上下文隔离的 Codex 审查，并由确定性门禁决定能否交付。
- 5. 审查不通过时自动返工，达到次数上限后转人工处理。
- 6. 任务通过后创建或更新 GitLab Merge Request，并继续执行下一项任务。
- 7. 手机 App 创建的任务和附件可在在线 Runner 上自动被发现和执行。
- 8. Codex 可读取授权的全项目上下文，但只能写入当前任务允许的子项目。
- 9. 跨团队修改需求被转化为依赖子任务，而不是临时扩大当前 Runner 的写权限。
- 10. 所有身份、状态、代码版本、附件版本、执行、审查、审批和失败均可追溯。

非目标

- 不重新实现 Codex 桌面应用、模型推理、聊天、终端或代码编辑器。
- 不替代 GitLab 的源码管理、Merge Request、CI/CD 和保护分支能力。
- 不把共享服务器文件夹作为源码的唯一真实来源。
- 首期不允许 Codex 自动合并主分支或自动部署生产环境。
- 首期不承诺所有任务均可完全无人干预；需求不清、高风险和跨团队冲突必须允许人工接管。
- 首期不建设复杂的多区域、高并发分布式调度和硬件设备池自动化。

成功标准

维度	成功标准	建议度量
自动化	用户无需逐条向 Codex 投递任务；队列可持续运行	自动链路完成率、人工介入率
安全	无非授权目录写入、无保护分支直接推送	安全事件数、越权拦截率
可追溯	每个任务均关联线程、Commit、MR、测试与审查	审计完整率
可靠性	Runner 中断后任务可恢复或安全重试	恢复成功率、重复有效执行率
协作	跨团队依赖形成子任务并可自动解除阻塞	依赖等待时长、闭环率
体验	手机创建任务后快速进入看板并被 Runner 发现	同步延迟、领取延迟

范围与假设

- 源码托管于公司 GitLab（可为自建 GitLab），大项目优先采用 Group 下的 frontend、backend、hardware、contracts 多仓库结构。
- 公司提供统一身份认证或可在 MVP 阶段提供最小可用账号系统。

- 公司任务服务器可对外提供 HTTPS API，并支持 SSE/WebSocket 或长轮询。
- 公司具备 MinIO/S3 等对象存储，或允许在 MVP 中部署兼容服务。
- 开发者电脑能够运行 Windows 后台服务和官方 Codex 运行时，并有受控 GitLab 网络访问。
- 本地代码执行要求电脑和 Runner 在线；电脑离线时任务保留在服务器队列。
- 插件可以提供自定义 UI，但任意插件是否能够永久注册 Codex 原生左侧一级导航项不作为已确认能力；任务中心必须有稳定的插件页面或独立 Web/Runner UI 入口。

用户角色与权限模型

角色	主要职责	默认代码权限
前端工程师	创建和执行前端任务；读取接口、协议和附件	frontend 读写；其他子项目只读
后端工程师	创建和执行后端任务；维护 API 和数据服务	backend 读写；其他子项目只读
硬件工程师	处理固件、协议、设备相关任务	hardware 读写；其他子项目只读
产品经理	创建需求、上传资料、查看进度和验收	通常无源码写权限
测试工程师	提交缺陷、复现资料和验收结果	测试资料与评论权限
代码审查人	审查 Diff、测试、CI 和验收证据	目标范围只读、评论和审批
项目负责人	管理优先级、依赖和最终审批	按项目策略授权
系统管理员	管理项目、角色、Runner 和安全策略	配置权限，不默认执行代码

身份与授权原则

授权公式 任务可执行范围 = 任务创建/审批权限 $\cap$ 项目角色策略 $\cap$ 当前任务范围 $\cap$ execution_principal 权限 $\cap$ Runner 设备能力与安全策略。

- 用户通过公司 SSO/OIDC 登录，服务器返回可访问项目、角色和策略；客户端不得自行声明角色。
- 一个用户可在不同项目拥有不同角色；切换角色必须重新获得服务器授权上下文。
- MVP 采用用户绑定的队列会话：登录用户选择项目和角色，退出或授权撤销后停止领取新任务；实际代码执行使用服务器签发的任务级执行授权。
- Runner 绑定组织、项目、角色队列、设备和能力标签；GitLab、对象存储和 Codex 凭证分别建模，不隐式复用当前 UI 登录用户身份。
- 任务创建时可由 UI 默认填写当前角色，但服务器必须重新验证 target_component 和 write_scopes。
- 用户可以查看授权范围内其他团队任务，但前端 Runner 只能领取 frontend 队列任务。
- 高风险审批需要再次确认用户身份，并把批准绑定到当前任务、当前产物哈希和有效期。

执行主体模型

主体	含义	权限来源
requester	创建任务的用户	SSO、项目成员关系和角色
task_owner / assigned_role	任务归属人与负责团队	任务服务器状态与项目策略

主体	含义	权限来源
runner_device	实际运行 Runner 的设备	设备注册、健康状态和能力标签
execution_principal	本次任务的代码执行主体	服务器签发的短期任务授权
git_transport_principal	仅推送任务分支的 Git 身份	目标项目、目标分支和短有效期
gitlab_api_principal	创建 MR、查询 Pipeline 的 API 身份	受控 Broker 或独立最小权限凭证
approver	审批高风险动作或业务门禁的用户	再认证、Code Owner 和审批策略

默认仓库权限矩阵

身份	Frontend	Backend	Hardware	Contracts
前端	读写任务分支	只读	只读	只读
后端	只读	读写任务分支	只读	按策略只读/写
硬件	只读	只读	读写任务分支	按策略只读/写
审查人	只读/审查	只读/审查	只读/审查	只读/审查
管理员	配置权限	配置权限	配置权限	配置权限

默认附件权限矩阵

身份	项目附件读取	上传	敏感附件审批
前端/后端/硬件	任务明确引用的版本	允许上传到有权项目	无
产品/测试	项目策略允许的附件	允许	无
审查人	审查任务所需附件	通常不允许	按策略
管理员/安全人员	按职责授权	按职责授权	允许

典型场景

场景 A：连续执行前端任务

- 11. 前端工程师登录并选择大项目与前端身份。
- 12. 用户一次创建 10 条前端任务，服务器分配队列序号。
- 13. 空闲 Runner 原子领取第一条 READY 任务，准备隔离工作区。
- 14. Codex 读取前端、后端、硬件、协议和附件，只修改前端。

15. 测试、Diff Guard 和上下文隔离的自动审查通过后创建 MR；MVP 默认在任务达到项目完成条件后推进队列。
16. Runner 立即创建新的 Codex thread 执行下一条任务。

场景 B：手机端创建任务

17. 用户在手机 App 选择项目，系统默认当前授权角色。
18. 用户填写任务说明、验收标准并上传截图、PDF 或日志。
19. 服务器保存任务和附件版本，通过事件通知在线 Runner。
20. Runner 在满足顺序、依赖和权限条件后自动执行。
21. 用户在手机端查看执行、审查、MR 和完成状态。

场景 C：发现跨团队依赖

22. 前端 Codex 发现后端接口缺少字段。
23. Codex 输出结构化 cross_team_request，而不是直接修改 backend。
24. 服务器创建后端子任务或等待人工确认，原前端任务进入等待依赖。
25. 后端任务完成后，原前端 attempt 结束；服务器解析新的后端 Commit、重算 input_hash，并创建新的前端 attempt。

场景 D：审查失败后返工

26. 自动审查发现测试覆盖或实现问题，任务进入 CHANGES_REQUESTED。
27. 审查意见发送回原执行线程进行返工，再次进入审查。
28. 超过最大返工次数后进入 NEEDS_ATTENTION，由人工决定继续、拆分或终止。

场景 E：Runner 中断恢复

29. Runner 掉线后服务器等待租约到期，任务进入恢复状态。
30. 原 Runner 恢复时根据 thread、worktree、Commit、租约 generation 和外部 Operation 对账。
31. 无法安全恢复时创建新 attempt；旧 attempt 分支和 MR 自动隔离，确保不会重复有效推送或交付。

任务中心 UI 需求

任务中心是用户查看任务、Runner 和 Codex 执行状态的主要入口，可由 Codex 插件自定义 UI、独立 Web/Runner UI 和移动端复用同一套服务端数据。四列看板保持业务层简洁，复杂恢复状态在任务卡和详情页中呈现，不额外增加主列。

Codex UI 边界 首期不依赖任意插件永久注册 Codex 原生左侧一级导航项。推荐优先交付插件自定义任务页面或全屏任务中心，并保留独立 Runner/Web UI；若阶段 0 验证存在稳定导航扩展点，再增加原生入口。

顶部状态区
└ 当前项目 / 当前身份 / Runner 在线状态
└ 自动执行开关 / 完成当前任务后暂停 / 紧急停止
└ 当前执行任务 / 队列数量 / 告警数量
└ 搜索 / 筛选 / 新建任务
四列看板
└ 待办
└ 执行
└ 审查
└ 完成
任务详情
└ 需求与验收标准
└ Codex 线程与执行日志
└ 文件 Diff / 测试 / CI / MR
└ 附件与依赖
└ 审查、审批和审计记录

顶部状态区

- 持续显示组织、项目、当前角色和用户，不允许将角色信息隐藏在设置页。
- 显示 Runner 在线/离线、版本、设备名、当前任务、租约剩余时间和最近心跳。
- 提供自动执行总开关、完成当前任务后暂停、紧急停止三个不同语义的控制项。
- 紧急停止必须阻止领取新任务，并请求停止当前 Codex turn；是否保留工作区由策略决定。
- 显示当前队列数量、等待依赖数量、待人工处理数量和过去 24 小时完成数量。
- 提供项目、角色、优先级、创建人、状态、时间和全文搜索筛选。

四列看板映射

业务列	内部状态	核心用户操作
-----	------	--------

业务列	内部状态	核心用户操作
待办	QUEUED、READY、BLOCKED_DEPENDENCY、RETRY_WAIT、PAUSED、SCHEDULED	新增、编辑、排序、暂停、取消、查看阻塞、指定下一项
执行	CLAIMED、PREPARING、RUNNING、WAITING_TOOL_APPROVAL、WAITING_EXTERNAL_RESULT、RECOVERING	查看日志、审批受控工具动作、暂停、停止、补充说明、人工接管
审查	AUTO_REVIEWING、CI_RUNNING、CHANGES_REQUESTED、HUMAN_REVIEW、WAITING_MERGE_APPROVAL、READY_TO_MERGE、NEEDS_ATTENTION	查看 Diff、批准、请求修改、重试、转交审查、创建/查看 MR
完成	SUCCEEDED、MERGED、CANCELED、REJECTED、SUPERSEDED	查看摘要、MR、审计、复制任务、重新打开

每个内部状态只映射到一个主列。执行阶段的工具审批使用 `WAITING_TOOL_APPROVAL`；合并、发布或业务验收审批使用 `WAITING_MERGE_APPROVAL`，禁止复用一個含义模糊的 `WAITING_APPROVAL`。

任务卡片

- 基础信息：任务编号、标题、所属子项目、角色、优先级、创建人和截止时间。
- 执行信息：Runner、Codex thread、已运行时长、当前步骤、最近事件和文件变更数。
- 审查信息：自动审查结果、CI、审批状态、返工次数和风险等级。
- 关联信息：依赖任务、附件数量、GitLab 分支、Commit 和 MR。
- 权限信息：当前读写范围、待审批动作和批准的有限操作；普通审批不得扩大到其他角色仓库。
- 异常标识：等待依赖、等待审批、Runner 离线、测试失败、越权修改和需要人工处理。

任务详情页

- 需求区：标题、详细说明、业务背景、技术约束、验收标准、Definition of Done。
- 上下文区：各仓库固定 Commit、contracts 版本、附件版本和依赖关系。
- 执行区：事件时间线、命令摘要、测试、修改文件、资源消耗和 Codex 摘要。
- 审查区：权限检查、测试证据、验收标准逐项结果、Codex 审查、人工评论和审批。
- GitLab 区：任务分支、Commit、MR、Pipeline 和 Code Owner 状态。
- 审计区：创建、领取、状态变更、凭证使用、审批、重试、取消与恢复。

移动端需求

- 支持公司账号登录、项目选择和授权角色切换。
- 支持创建、编辑、排序和取消尚未执行的任务。
- 任务创建必须支持多条验收标准和任务模板。
- 支持上传图片、PDF、日志、视频和录音等附件，并展示扫描状态。
- 支持查看四列状态、执行摘要、审查意见、MR 和 CI。
- Pilot 阶段支持批准、请求修改和紧急暂停；高风险审批必须再次认证。
- 离线时允许保存任务草稿，恢复网络后再提交，不得静默重复创建。

任务状态机

任务主状态与自动循环



图 1 任务四列映射、返工回路与可配置队列推进点

主流程

```
QUEUED → READY → CLAIMED → PREPARING → RUNNING
      → AUTO_REVIEWING → CI_RUNNING → HUMAN_REVIEW_REQUIRED (按策略)
      → READY_TO_MERGE → SUCCEEDED / MERGED
```

异常与恢复流程

```
RUNNING → RETRY_WAIT → READY
RUNNING → RECOVERING → RUNNING / NEEDS_ATTENTION
AUTO_REVIEWING → CHANGES_REQUESTED → RUNNING
QUEUED / RUNNING → BLOCKED_DEPENDENCY → READY
```

任意非终态 → PAUSED → 原状态 / READY

任意非终态 → CANCELED

自动交付就绪条件

- Codex 执行正常结束并生成结构化执行结果。
- 最终 Diff 未超出 write_scopes，且不存在绝对禁止动作或未解决的安全告警。
- 任务声明的本地必需测试通过，测试在隔离执行器内完成。
- 上下文隔离的自动 Codex 审查通过，且 Diff Guard、Secret Scan 等确定性门禁通过。
- 每条验收标准均有通过、失败或无证据的结构化结论，成功任务不得存在失败项。
- 任务分支、Commit、MR、附件版本、执行摘要和逻辑追加写审计事件已持久化。

业务完成条件

- 项目策略要求的 GitLab Pipeline、Code Owner 和人工审批均已满足。
- MR 已达到可合并状态或已经合并，具体完成点由 task_completion_policy 决定。
- GitLab 实际状态已通过 Webhook 与周期性 reconciliation 对账，任务服务器不存在未解决的不一致。
- 执行租约已释放，临时凭证已失效，工作区进入保留或清理策略。

队列推进策略 MVP 默认 queue_advance_policy = AFTER_TASK_COMPLETE，严格满足“完成当前任务再执行下一项”。Pilot 可选择 AFTER_DELIVERY_READY：自动审查通过并创建 MR 后释放 Runner 执行槽，但原任务仍停留在审查列等待 CI、Code Owner 或人工审批。

详细功能需求

优先级 P0 = MVP 必须实现；P1 = Pilot 优先实现；P2 = 后续增强。所有 P0 条目都应有自动化或可重复的验收用例。

登录、身份与 Runner 注册

FR-AUTH-001 [P0] 系统必须支持公司统一身份认证，并从服务器获取用户可访问的组织、项目和角色。

FR-AUTH-002 [P0] 用户必须明确选择当前项目和当前工作身份，任务中心持续显示该上下文。

FR-AUTH-003 [P0] 客户端提交的角色、project_id、write_scopes 不得直接作为授权依据，服务器必须重新计算。

FR-AUTH-004 [P0] Runner 必须注册为独立设备，获得可撤销的短期凭证并定期刷新。

FR-AUTH-005 [P0] 用户退出、权限撤销或设备吊销后，Runner 不得领取新任务。

FR-AUTH-006 [P0] Runner UI 必须展示设备名、版本、在线状态、心跳、当前项目和角色。

FR-AUTH-007 [P1] 支持用户在不同项目拥有不同角色，并允许授权范围内切换。

FR-AUTH-008 [P1] 高风险审批必须重新认证，并把批准绑定到产物哈希和有效期。

FR-AUTH-009 [P0] 系统必须分别记录 requester、assigned_role、runner_device、execution_principal、GitLab 主体和 approver，不得把当前 UI 登录用户隐式当作所有外部系统身份。

项目、子项目与策略

FR-PROJ-001 [P0] 系统支持一个大项目关联 frontend、backend、hardware、contracts 等多个 GitLab 仓库。

FR-PROJ-002 [P0] 管理员可为每个角色配置默认可读仓库、可写仓库、任务命令网络策略、允许的受控服务和测试命令。

FR-PROJ-003 [P0] 任务必须保存所有相关仓库的固定 Commit SHA，执行期间不得被最新分支静默替换。

FR-PROJ-004 [P0] 每个仓库维护自身 AGENTS.md；跨仓库公共规则由 Runner 版本化后显式注入或复制到本次目标仓库根，不得假定兄弟仓库 AGENTS.md 自动生效。

FR-PROJ-005 [P0] 前端身份默认只可写 frontend、backend、hardware、contracts 和附件仅可读。

FR-PROJ-006 [P1] 项目支持任务模板、验收标准模板、分支命名规则、MR 模板和 Code Owner 策略。

FR-PROJ-007 [P1] 项目可配置维护窗口、最大并发、单任务时间/费用预算和自动执行时段。

任务创建、编辑与排序

FR-TASK-001 [P0] 用户可从 Codex 任务中心、Web 和手机 App 创建任务。

FR-TASK-002 [P0] 用户可以连续创建任意数量的待办；服务器为同一队列分配稳定 queue_sequence。

FR-TASK-003 [P0] 任务至少包含标题、详细说明、目标子项目和一条验收标准。

FR-TASK-004 [P0] 任务支持优先级、手工排序、截止时间、依赖、附件、技术约束和必需测试。

FR-TASK-005 [P0] 尚未领取的任务可编辑、排序、暂停和取消；关键字段变更必须保留历史。

FR-TASK-006 [P0] 服务端必须拒绝目标子项目或 write_scopes 超出用户权限的任务。

FR-TASK-007 [P1] 支持从模板、历史任务和 GitLab Issue 创建任务。

FR-TASK-008 [P1] 支持父子任务、依赖图、定时执行和完成后重新打开。

FR-TASK-009 [P2] 支持从文档或语音生成任务草稿，但正式提交前必须人工确认。

任务看板与实时状态

FR-BOARD-001 [P0] 任务中心必须提供待办、执行、审查和完成四列。

FR-BOARD-002 [P0] 内部状态必须由服务器映射到唯一业务列，客户端不得自行改变主状态。

FR-BOARD-003 [P0] 状态变化应通过 SSE/WebSocket 推送，轮询作为降级；正常网络下 5 秒内可见。

FR-BOARD-004 [P0] 执行卡显示 Runner、thread、已运行时间、当前步骤、最近日志和变更文件数。

FR-BOARD-005 [P0] 审查卡显示测试、CI、自动审查、人工审批和返工次数。

FR-BOARD-006 [P0] 失败、等待依赖、等待审批、Runner 离线和越权修改必须醒目标记。

FR-BOARD-007 [P1] 支持全文搜索、保存个人视图、依赖图和统计趋势。

队列与自动调度

FR-SCHED-001 [P0] Runner 自动获取当前项目和角色的可执行任务，默认同时只执行一项。

- FR-SCHED-002 [P0]** 任务领取必须为服务器端原子操作，并创建唯一 active_run 和租约 generation。
- FR-SCHED-003 [P0]** 只有依赖已完成、权限合法、附件可用且没有有效租约的任务可被领取。
- FR-SCHED-004 [P0]** 同一任务不得存在两个有效执行；服务端状态更新使用 fencing token，Commit、Push、MR 等外部副作用执行前必须重新校验租约并取得任务级短期操作授权。
- FR-SCHED-005 [P0]** 达到 queue_advance_policy 指定的推进点后，Runner 应在 30 秒内尝试领取下一条 READY 任务；MVP 默认推进点为任务业务完成。
- FR-SCHED-006 [P0]** 每个新任务必须创建新的 Codex 执行线程；不得继承上一任务上下文。
- FR-SCHED-007 [P0]** 系统提供自动执行开关、完成当前任务后暂停和紧急停止。
- FR-SCHED-008 [P0]** Runner 定期续租；租约过期后任务进入 RECOVERING，不得立即盲目重放。
- FR-SCHED-009 [P1]** 支持 Runner 能力标签、指定 Runner、维护窗口和每队列并发限制。
- FR-SCHED-010 [P2]** 支持多 Runner 并发、优先级抢占和资源感知调度。
- FR-SCHED-011 [P0]** Task 必须记录 task_completion_policy 和 queue_advance_policy，Run 必须记录 execution_slot_released_at，UI 清楚区分自动交付就绪与业务完成。
- FR-SCHED-012 [P0]** 任务进入 NEEDS_ATTENTION 时 MVP 默认暂停所属队列；只有授权人员明确选择重试、跳过或终止后才能继续，Pilot 可配置其他 error_advance_policy。

Codex 执行与工作区

- FR-EXEC-001 [P0]** Runner 必须调用 OpenAI 官方 Codex SDK 或本地 App Server/CLI 运行时；本地运行时再访问 OpenAI 服务，不得把 App Server 描述成公司外部托管业务服务。
- FR-EXEC-002 [P0]** 每个任务使用独立工作区、任务分支和固定输入快照。
- FR-EXEC-003 [P0]** Codex 可读取授权上下文，但只可写入 write_scopes 和 Runner 输出目录。
- FR-EXEC-004 [P0]** 执行前校验仓库 Commit、附件哈希、App Server sandboxPolicy、任务命令网络策略和测试配置；网络域名限制由出口代理、防火墙或容器策略实施。
- FR-EXEC-005 [P0]** 执行中持续上报结构化事件，UI 可查看脱敏后的实时或准实时日志。
- FR-EXEC-006 [P0]** 执行后必须再次检查最终 Diff、软链接、路径穿越和越权目录变更。
- FR-EXEC-007 [P0]** 必须在与 Codex 等同或更严格的受限环境运行构建、Lint、测试、依赖安装和附件解析；不得继承 Runner 服务凭证或完整环境变量。
- FR-EXEC-008 [P0]** Runner 负责生成 Commit，并通过受控 GitLab Broker 或短期凭证推送任务分支、创建 MR；Git 传输凭证与 MR API 凭证分离且不进入 Codex/测试子进程。

FR-EXEC-009 [P0] 失败必须记录阶段、错误分类、可恢复性、线程和工作区信息。

FR-EXEC-010 [P1] 支持恢复原线程和工作区、用户补充非冲突说明、资源成本统计和超预算熔断。

审查、返工与审批

FR-REVIEW-001 [P0] 执行完成后自动进入审查，自动 Codex 审查必须使用与执行上下文隔离的新上下文；该隔离不等同于独立安全主体。

FR-REVIEW-002 [P0] 自动审查覆盖权限边界、Diff、测试、CI、验收标准和跨项目影响。

FR-REVIEW-003 [P0] 审查结论必须结构化为通过、需要修改或需要人工处理，并逐条关联证据。

FR-REVIEW-004 [P0] 审查不通过时可把意见发送回原执行线程；最大自动返工次数可配置。

FR-REVIEW-005 [P0] 超过返工上限后进入 NEEDS_ATTENTION，停止自动循环并通知负责人。

FR-REVIEW-006 [P0] CI 未满足项目规则时任务不得标记成功。

FR-REVIEW-007 [P0] 高风险操作需要授权人员批准，批准必须记录主体、范围、哈希、时间和意见。

FR-REVIEW-008 [P1] 支持 Code Owner、多级审批和安全/性能/兼容性专项模板。

跨团队依赖

FR-DEP-001 [P0] 任务可声明多个前置依赖；依赖未完成时不得执行。

FR-DEP-002 [P0] Codex 可以输出结构化 cross_team_request，包含目标团队、原因、证据和验收标准。

FR-DEP-003 [P0] 是否自动创建跨团队子任务由项目策略控制；Pilot 默认需要人工确认。

FR-DEP-004 [P0] 跨团队请求不得临时扩大当前 Runner 写权限。

FR-DEP-005 [P0] 依赖完成后不得原地修改活动 Run 的固定输入；服务器结束旧 attempt、解析新 Commit、重算 input_hash，并创建新的 attempt。

FR-DEP-006 [P1] 系统检测循环依赖，并支持看板依赖图和阻塞链。

附件功能需求

FR-ATT-001 [P0] 附件必须存储在受控对象存储，支持任务级和项目公共附件。

FR-ATT-002 [P0] 每个附件保存版本、上传者、SHA-256、类型、大小、访问范围和扫描状态。

FR-ATT-003 [P0] Runner 只下载任务明确引用且用户有权访问的附件。

FR-ATT-004 [P0] 任务开始时固定附件版本，执行中的更新不得静默替换输入。

FR-ATT-005 [P0] 附件默认只读挂载，并限制大小、类型、解析方式和网络行为。

FR-ATT-006 [P0] 附件和源码中的文字视为不可信数据，不得覆盖系统指令或权限策略。

FR-ATT-007 [P1] 支持预览、版本对比、文本提取、归档和生命周期策略。

GitLab 集成

FR-GIT-001 [P0] GitLab 是源码、分支、Commit、MR 和 CI 的唯一真实来源。

FR-GIT-002 [P0] 每项任务创建独立任务分支，命名包含 task_key 和 attempt。

FR-GIT-003 [P0] MVP/Pilot 中 Runner 账号不得直接推送或合并受保护分支；目标分支必须受保护，并将 Runner 的直接 Push/Merge 权限设为无。

FR-GIT-004 [P0] 任务执行通过后创建或更新 MR，并关联任务、测试、审查和执行摘要。

FR-GIT-005 [P0] GitLab Webhook 用于加速同步 MR、Pipeline、审批和合并状态；服务器还必须周期性 reconciliation，修复漏发、乱序和处理失败。

FR-GIT-006 [P0] 最终完成条件服从保护分支、Code Owner、审批和 CI 规则；Code Owner approval 必须在目标 GitLab 版本、许可和项目配置中实际启用。

FR-GIT-007 [P1] 支持 GitLab Issue 双向同步、多实例和项目级 MR 模板。

通知、管理与审计

FR-ADMIN-001 [P0] 管理员可配置项目、角色、仓库、读写范围、测试、审查和高风险规则。

FR-ADMIN-002 [P0] 所有状态迁移、权限拒绝、审批、命令摘要、凭证使用和外部操作写入审计日志。

FR-ADMIN-003 [P0] 审计采用逻辑追加写，普通用户不得修改；事件包含 trace_id 和前后状态。只有启用哈希链、独立权限、远端归档或对象锁后才可称防篡改。

FR-ADMIN-004 [P0] 管理员可暂停单任务、队列、项目或全部 Runner，并撤销设备凭证。

FR-NOTIFY-001 [P0] 任务开始、进入审查、需要审批、失败、依赖解除和完成时通知相关用户。

FR-NOTIFY-002 [P0] Runner 离线、租约异常、越权修改和成本异常必须产生告警。

FR-ADMIN-005 [P1] 支持按任务、用户、Runner、项目检索审计，导出报告并统计周期、失败率和成本。

绝对禁止动作（MVP / Pilot）

- 直接推送或合并受保护分支，以及由 Runner 绕过 GitLab 审批规则执行合并。

- 写入非当前任务目标仓库、临时扩大为其他角色仓库写权限，或启用 danger-full-access/关闭核心沙箱。
- 自动发布生产环境、执行生产破坏性数据库变更、删除或覆盖不可恢复的业务数据。
- 修改权限、密钥、保护分支、Code Owner 或 CI/CD 安全门禁。
- 向未批准的外部网络上传源码、日志、附件或凭证。

可进入有限审批的动作

- 在目标仓库和受控测试环境内运行需要网络的依赖安装、构建或集成测试。
 - 访问已登记的公司 Git、包代理、测试服务或设备仿真服务。
 - 在目标仓库内修改被标记为高风险但允许变更的文件，例如迁移脚本、依赖锁文件或受控构建配置。
 - 连接受控实验室设备执行可回滚测试；真实固件烧录和设备控制需独立设备安全流程。
32. Runner 暂停相关步骤并将任务移动到审查/审批状态。
 33. UI 展示拟执行动作、影响范围、文件/资源、产物哈希和风险。
 34. 授权人员批准或拒绝；批准只对当前任务、当前动作和限定时间有效。
 35. 执行前再次验证任务租约和产物哈希；变化后原批准失效。
 36. 审批与实际执行结果写入逻辑追加写审计；普通用户不可修改。

总体技术架构

系统总体架构

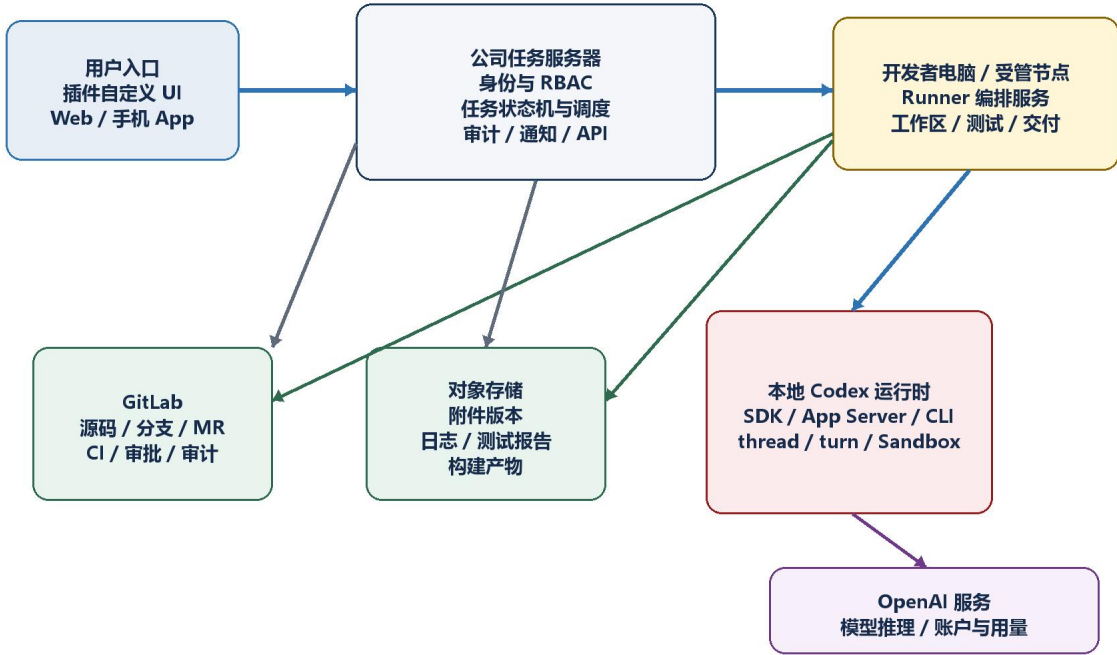


图 2 系统控制面、执行面、数据面与 Codex 调用链

整体采用控制面、执行面和数据面分离的架构。公司任务服务器是控制面，负责身份、策略、任务状态机、租约和审计；本地 Runner 是执行编排器，在开发者电脑或受管节点上启动官方 Codex SDK/App Server/CLI 运行时并准备工作区；本地 Codex 运行时再访问 OpenAI 服务。GitLab 与对象存储构成数据面。插件 UI、Web 和手机 App 都只是受控客户端，不负责维持后台执行。

架构原则

- 职责分离：UI 不承担后台循环；Runner 不成为任务或源码的唯一事实来源。
- 零信任任务内容：任务描述、源码、附件和日志都不能扩大系统权限。
- 最小权限：Codex 子进程、Runner、Git 传输凭证、GitLab API 凭证、对象存储凭证和任务命令网络访问均按任务最小化。
- 可恢复：所有重要阶段有持久化运行记录、租约、幂等键和对账信息。
- 可替换：Codex SDK/App Server 通过适配层接入，外部接口变化不直接扩散到业务层。

- 可审计：用户、Runner、Codex、GitLab、CI、审批人等主体均写入统一的逻辑追加写审计流。
- 固定输入：源码 Commit、附件版本、策略版本、提示模板和模型配置在单次运行内不可漂移。

组件职责

组件	主要职责	持久化数据
Codex 插件 UI / Web	四列看板、任务详情、登录上下文、审批入口	不作为事实源；仅本地 UI 偏好
手机 App	创建任务、上传附件、查看状态和通知	本地草稿；正式数据在服务器
任务 API	项目、用户、角色、任务、依赖、状态、通知和管理接口	PostgreSQL
调度服务	原子领取、租约、重试、队列顺序、恢复和熔断	Task、Run、Lease、Operation
Runner	登录、心跳、工作区、Codex 调用、测试、GitLab 和结果回传	本地临时缓存与恢复清单
Codex 适配层	统一本地 SDK/App Server 调用、事件、审批、取消和审查	thread_id、turn_id、配置版本
本地 Codex 运行时	CLI/App Server、thread/turn、工具调用和 Sandbox	本地会话状态与受控工作区
OpenAI 服务	模型推理、账户认证、配额和用量	由 OpenAI 服务管理
GitLab	源码、分支、Commit、MR、CI、Code Owner 和保护分支	Git 与 GitLab 数据
对象存储	附件、日志、测试报告、构建产物和审查报告	不可变对象与版本
审计/监控	事件检索、指标、告警、成本和安全检测	日志、指标、Trace

插件 UI 与后台 Runner 的边界

边界结论 插件 UI 负责展示和用户操作，不能作为持续运行的后台调度器；手机任务到达后真正唤醒执行的是常驻 Runner。

- 推荐由插件 UI 调用声明给组件的 MCP 工具访问公司任务服务；若 iframe 直接访问公司 API，必须配置允许连接域、短期会话与独立 OAuth，禁止保存 Runner 或 GitLab 长期凭证。
- 插件可以触发创建、暂停、取消、批准和请求修改，但所有写操作都由服务器校验。
- 插件公开能力包括自定义 UI，但任意插件永久注册原生左侧一级导航项不是 MVP 的硬依赖；阶段 0 只把它作为兼容性验证项。

- Runner 以 Windows Service、受管后台进程或托盘程序运行，即使任务中心 UI 未打开仍可工作。
- Runner 使用官方 Codex SDK 作为自动化入口；需要完整 thread/turn 事件、review/start 和 sandboxPolicy 时启动本地 App Server。
- 为避免官方接口变化，业务层仅调用内部 CodexAdapter 接口，不直接依赖 SDK/App Server 数据结构。

Codex 集成方案

集成目标

- 使用 OpenAI 官方 Codex 作为代码执行与自动审查引擎；Runner 不重新实现模型、对话、代码理解或工具循环。
- 每个任务创建独立执行 thread；同一任务返工可继续原执行 thread。
- 自动审查使用新上下文或 App Server review/start，与执行上下文隔离；该隔离不能替代测试、CI、Diff Guard、Code Owner 和人工审批。
- Runner 订阅流式事件、完成、错误、审批和取消结果，并转换为内部标准事件。
- SDK/App Server 版本必须固定，升级前运行契约测试与 Codex 质量评测集。

CodexAdapter 建议接口

```
interface CodexAdapter {  
  startExecution(input: ExecutionInput): Promise<ExecutionHandle>;  
  resumeExecution(threadId: string, input: FollowUpInput): Promise<ExecutionHandle>;  
  streamEvents(handle: ExecutionHandle): AsyncIterable<CodexEvent>;  
  startReview(input: ReviewInput): Promise<ReviewHandle>;  
  cancel(handleId: string): Promise<void>;  
  respondToApproval(requestId: string, decision: ApprovalDecision): Promise<void>;  
  getThread(threadId: string): Promise<ThreadSnapshot>;  
}
```

兼容性注意 官方公开能力支持 thread/turn 与自定义插件 UI，但后台进程创建的 thread 是否实时出现在当前桌面应用侧栏、以及任意插件能否注册永久一级导航入口，都不能作为核心业务保证。任务中心必须保存 thread_id、摘要和事件，并提供稳定的插件页面或独立 UI。

Runner 架构

Runner 是一个薄型常驻编排服务，不包含模型能力。它从公司服务器领取任务，准备固定版本工作区，调用官方 Codex，执行测试和安全检查，然后把代码交付到 GitLab。

Runner 内部模块

模块	职责
Auth Client	SSO/device 登录、短期令牌刷新、设备吊销处理
Queue Client	长轮询/SSE/WebSocket、claim、续租、状态更新和幂等调用
Workspace Manager	Git 缓存、worktree、固定快照、只读挂载、Git hooks/filter/submodule/LFS 策略与清理
Policy Engine	计算有效权限、任务命令网络策略、命令规则、绝对禁止项和审批门禁
Codex Adapter	SDK/App Server 调用、事件归一化、取消和恢复
Isolated Test Executor	在受限令牌、容器/WSL 或等效沙箱中运行构建、Lint、依赖安装与测试，不继承 Runner 凭证
Diff Guard	检查最终 Diff、越权路径、软链接、敏感信息和文件类型
GitLab Client / Broker	分离 Git Push 与 MR API 权限，执行租约复核、短期操作授权、幂等交付和状态对账
Recovery Manager	启动对账、租约恢复、孤儿进程清理和现场保留
Telemetry	日志脱敏、Trace、指标、成本和告警

Runner 执行算法

37. 验证 requester 会话、项目角色、runner_device、执行主体和 Runner 版本，建立出站长连接或轮询。
38. 调用 claim-next-task，由服务器原子选择 queue_sequence 最小的 READY 任务并返回租约。
39. 校验任务权限、依赖、预算、目标仓库、固定 Commit、附件和项目策略。
40. 创建任务工作区：目标仓库可写 worktree，其他仓库和附件为只读快照；禁用或约束 Git hooks、外部 filter、子模块和 LFS smudge。
41. 启动新的 Codex 执行 thread，传入任务、验收标准、目标仓库 AGENTS、显式版本化跨仓库上下文和 App Server sandboxPolicy。
42. 消费事件并续租；危险动作转审批，取消时终止 turn 并停止后续副作用。
43. 在隔离测试执行器内运行构建、Lint 和测试，再执行最终 Diff 与敏感信息检查，生成结构化执行结果。
44. 启动上下文隔离的自动 Codex 审查；失败则按策略返工，超过次数转人工。
45. 通过后生成 Commit；Push 与创建/更新 MR 前重新校验 lease_generation，并从服务器取得任务级短期操作授权。

46. 把自动交付、GitLab 关联和逻辑追加写审计持久化；按 `queue_advance_policy` 释放执行槽并领取下一项，业务完成状态由 CI/审批/合并对账推进。

Runner 伪代码

```
while runner.enabled:
    task, lease = server.claim_next(project, active_role, runner_id)
    if task is None:
        await wait_for_event_or_timeout()
        continue

    workspace = prepare_isolated_workspace(task)
    execution = codex.start_execution(task, workspace, sandbox_policy)
    result = await monitor_with_lease(execution, lease)
    test_result = isolated_tests.run(task, workspace)
    guard_result = validate_diff_and_secrets(task, workspace)
    review = codex.start_context_isolated_review(task, result, test_result, guard_result)

    if review.approved:
        commit = create_commit(task, workspace)
        op_auth = server.authorize_gitlab_operation(task.run_id, lease.generation, commit.hash)
        delivery = gitlab_broker.push_and_create_mr(op_auth, commit)
        server.mark_delivery_ready(task, result, review, delivery)
        server.release_slot_if_policy_allows(task)
    elif task.review_attempts < task.max_review_iterations:
        codex.resume_execution(task.execution_thread_id, review.findings)
    else:
        server.needs_attention(task, review.findings)
```

租约、幂等与恢复

- `claim` 必须在数据库事务内完成；单个严格串行队列只允许一个 `active_run`。
- 建议租约 60–120 秒，Runner 每 20–30 秒续租；更新同时校验 `task.version` 和 `lease_generation`。
- 无法保证绝对 `exactly-once`，应采用至少一次投递 + 幂等副作用。
- 外部操作使用稳定 `operation_key`，例如 `task/run/step`；超时后先查询结果再决定重试。
- 数据库状态更新携带 `fencing token`；仅依赖停止旧进程不能阻止网络分区下的 GitLab 副作用。
- Commit、Push、创建/更新 MR 前必须在线复核租约，并使用一次性 `operation authorization`、短期凭证或受控 GitLab Broker。
- 分支名包含 `run_id/attempt`；失效 `attempt` 的分支和 MR 自动隔离，不得被视为当前有效交付。
- 重启时先对账 Codex thread、工作区、Commit、MR、`operation_key` 和 GitLab 实际状态，再恢复或创建新 `attempt`。
- 瞬时网络错误指数退避；权限、需求不清、确定性测试失败和安全告警不自动重试。

GitLab 与源码管理

推荐仓库结构 使用一个 GitLab Group 管理 frontend、backend、hardware、contracts 和 project-docs 多仓库。对高度耦合团队可保留 monorepo，但仍应通过工作区和操作系统权限实现分区写入。

```
GitLab Group: smart-device
├─ frontend
├─ backend
├─ hardware
├─ contracts
└─ project-docs
```

GitLab 交付流程

47. Task 保存期望分支或版本约束；领取时服务器把所有相关仓库解析为不可变 Commit SHA，并写入本次 Run.base_refs 与 input_hash。
48. Runner 从缓存 fetch 固定 Commit，为目标仓库创建包含 task_key、run_id 和 attempt 的任务分支与 worktree。
49. 其他仓库以只读快照提供给 Codex，不指向成员的实时工作目录。
50. Codex 完成后 Runner 验证 Diff，只提交 write_scopes 内的文件。
51. Push 前重新校验租约；使用仅限目标项目和任务分支的短期 Git 传输凭证，禁止直接推送保护分支。
52. 通过独立 MR API 凭证或受控 Broker 创建 MR，写入任务链接、执行摘要、测试结果、审查报告和附件证据。
53. GitLab CI、Code Owner、审批和合并事件通过 Webhook 加速回写，服务器周期性 reconciliation 作为最终一致性兜底。
54. task_completion_policy 决定 MR 创建、CI 通过、可合并或实际合并中的业务完成点；queue_advance_policy 独立决定何时释放 Runner 执行槽。

分支与 Commit 规范

分支: codex/<role>/<task-key>-a<attempt>

示例: codex/frontend/FE-1024-a1

Commit: <task-key>: <summary>

示例: FE-1024: add device status panel

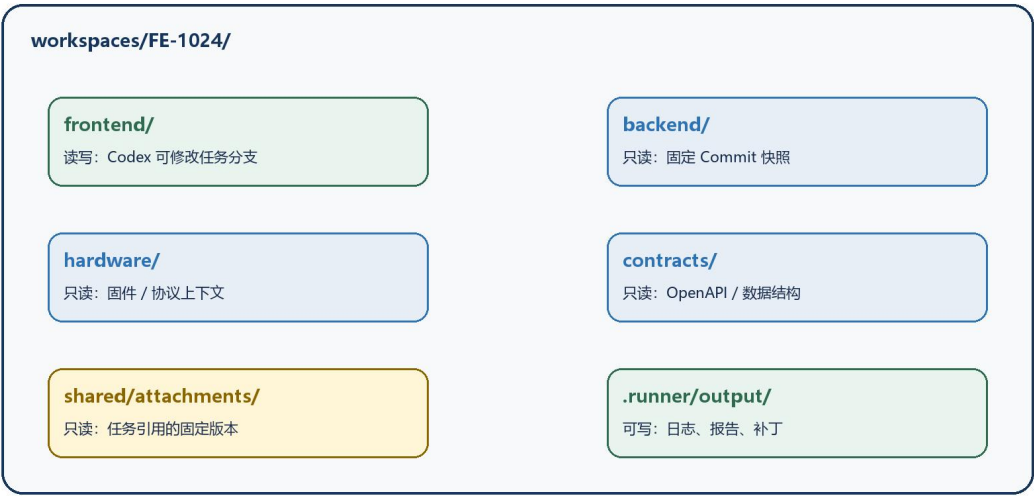
MR 标题: [FE-1024] Add device status panel

- 保护分支必须显式禁止 Runner 直接 Push/Merge；自动合并 MVP/Pilot 默认关闭。

- Code Owners 根据目录匹配前端、后端、硬件或 contracts 负责人；必须确认目标 GitLab 版本/许可支持，并在保护分支上启用 Code Owner approval。
- Git 传输凭证与 GitLab MR API 凭证分离，按目标项目和操作轮换，不进入 Codex、构建或测试子进程环境。
- Webhook 按实例能力验证签名或 X-Gitlab-Token，并校验来源、事件唯一标识和防重放条件。
- 大型、需要跟随版本的二进制可使用 Git LFS；高频日志、视频和构建产物进入对象存储。

任务工作区与权限隔离

前端身份的任务工作区与权限



权限原则: 登录用户权限 $\cap$ 项目角色权限 $\cap$ 当前任务范围 $\cap$ Runner 安全策略

图 3 前端身份: 目标仓库可写, 其他仓库与附件只读

每个 Task Run 必须在隔离目录执行，不得使用开发者日常工作目录或服务器共享源码目录。输入必须是固定 Commit 与固定附件版本，确保可重放、可审计和不受他人未提交变化影响。

```
workspaces/FE-1024/run-01/
├ frontend/           # 可写 worktree
├ backend/            # 固定 Commit, 只读
├ hardware/           # 固定 Commit, 只读
├ contracts/          # 固定 Commit, 只读/策略控制
├ shared/attachments/ # 当前任务附件, 只读
└ .runner/
  └ task.json         # 只读输入
```

└─ policy.json	# 只读策略
└─ output/	# 可写日志、补丁和报告

权限实施层次

层次	作用	说明
服务器 RBAC	决定用户、项目、角色和任务范围	禁止客户端伪造角色和 write_scopes
Runner Policy	计算本次运行的有效读写、网络 and 命令策略	策略版本写入 Run
Codex Sandbox	限制 Codex 进程的文件和网络访问	cwd 指向目标子项目，其他目录只读
OS/容器权限	高保证只读挂载和路径隔离	防止软链接、子进程或工具绕过
GitLab 权限	阻止直接修改主分支和未授权项目	Git Push 与 MR API 主体分离，凭证短期化
Diff Guard	执行后验证实际变更范围	发现越权时拒绝 Commit/Push

重要 AGENTS.md 只负责告诉 Codex 架构、命令和团队规范，不是安全边界。即使提示中写明“不要修改后端”，仍必须依赖 sandbox、操作系统权限和最终 Diff 检查。

App Server turn/start Sandbox 示例

以下仅作为 App Server turn/start 的结构示例；SDK 配置应由 CodexAdapter 单独映射，不能把同一 JSON 宣称为 SDK 与 App Server 的通用配置。所有路径在 Runner 解析后必须为绝对路径。

```
{
  "method": "turn/start",
  "params": {
    "threadId": "thr_FE-1024_a1",
    "cwd": "D:\\runner\\workspaces\\FE-1024\\run-01\\frontend",
    "sandboxPolicy": {
      "type": "workspaceWrite",
      "writableRoots": [
        "D:\\runner\\workspaces\\FE-1024\\run-01\\frontend",
        "D:\\runner\\workspaces\\FE-1024\\run-01\\.runner\\output"
      ],
      "readOnlyAccess": {
        "type": "restricted",
        "readableRoots": [
          "D:\\runner\\workspaces\\FE-1024\\run-01\\backend",
          "D:\\runner\\workspaces\\FE-1024\\run-01\\hardware",
          "D:\\runner\\workspaces\\FE-1024\\run-01\\contracts",
          "D:\\runner\\workspaces\\FE-1024\\run-01\\shared\\attachments"
        ],
        "includePlatformDefaults": true
      },
      "networkAccess": false
    }
  }
}
```

- 若平台或 Git 工具会把整个仓库根识别为可写工作区，应使用容器/WSL/ACL 做更强的只读挂载。
- Git Commit 与 Push 可由 Runner 自身在 Codex 退出后完成，避免为 Codex 开放 Git 元数据写权限。
- 执行前和执行后都检查真实路径、符号链接、大小写归一化、子模块、Git hooks、外部 filter、LFS smudge 和工作区外文件访问。
- sandboxPolicy.networkAccess 只表达任务命令网络开/关，不是域名白名单；域名限制由出口代理、防火墙或容器网络策略实施。
- 任务命令网络默认关闭；本地 Codex 客户端访问 OpenAI 的控制面网络与任务命令网络分开治理。

AGENTS.md 注入与仓库边界

```
workspaces/FE-1024/run-01/
├ frontend/AGENTS.md      # cwd 位于 frontend 时自动发现并生效
├ backend/AGENTS.md       # 只读资料，不假定自动生效
├ hardware/AGENTS.md      # 只读资料，不假定自动生效
├ contracts/AGENTS.md     # 只读资料，不假定自动生效
└ .runner/context.md      # Runner 显式注入、版本化的跨仓库公共规则
```

- Codex 的 AGENTS.md 自动发现沿项目/仓库根到当前工作目录路径进行；目标 cwd 位于 frontend 时，不假定兄弟 backend/hardware 仓库的 AGENTS.md 自动成为有效指令。
- frontend/AGENTS.md 提供本地启动、Lint、测试、构建和目录约定，并与 frontend Commit 一起固定。
- 跨仓库架构、统一 Definition of Done 和禁止行为由 Runner 生成版本化 context.md 并显式传入，或复制到本次目标仓库根。
- backend/hardware/contracts 的 AGENTS.md 可以作为只读参考资料，但不授予写权限，也不自动覆盖目标仓库规则。
- AGENTS、显式上下文、提示模板和项目策略必须版本化；Run 记录具体版本、Commit 或哈希。

附件存储与公共资料架构

用户界面可展示为“公共附件文件夹”，但后台不应把一个可变共享目录永久暴露给所有任务。附件应保存到对象存储，任务引用明确版本和哈希，Runner 只把当前任务授权使用的对象下载为只读快照。

附件数据要求

字段	说明
attachment_id	附件逻辑 ID
project_id / visibility	所属项目和访问范围
version	不可变版本号
object_key	对象存储位置
sha256	完整性校验和输入固定
mime_type / size	类型、大小与解析策略
uploaded_by / created_at	上传主体和时间
scan_status	病毒、恶意内容和敏感信息扫描结果
classification	公开、项目内、受限、敏感等分类
retention_policy	保留、归档和删除策略

- 附件扫描未通过、格式不允许或超出大小限制时不得进入执行环境。
- 任务执行过程中，附件新版本不得自动替换；用户可选择重启任务采用新版本。
- PDF、图片、日志等解析失败时必须给出明确错误，不得静默忽略。
- 附件中的任何指令都视为业务数据，不能覆盖系统策略、权限或审批规则。
- 项目公共附件仍应限制在当前项目，不应默认向整个公司所有任务开放。

数据模型

核心实体

实体	关键字段	职责
Organization / Project	id、name、policy_version	组织和大项目策略
Membership	user_id、project_id、roles、status	项目成员与角色授权
Repository	component、gitlab_project、default_branch	子项目与 GitLab 映射
Task	task_key、component、description、status、queue_sequence	用户可见任务
TaskDependency	task_id、depends_on、type	依赖关系与阻塞
ExecutionPrincipal	requester、device、execution、git_transport、gitlab_api	区分用户、设备和外部系统主体
Run	attempt、runner、thread、lease、base_refs、result	一次实际执行

实体	关键字段	职责
Review	run_id、decision、findings、artifact_hash	自动/人工审查
Approval	action_hash、scope、approver、expiry	高风险授权
AttachmentVersion	object_key、version、sha256、scan_status	固定附件输入
Operation	operation_key、request_hash、result_hash	外部副作用幂等
AuditEvent	actor、event_type、from/to、trace_id、prev_hash	逻辑追加写；可选哈希链与远端归档

Task 建议字段

字段组	字段
标识	id、task_key、organization_id、project_id、parent_task_id
业务	title、description、business_context、acceptance_criteria、definition_of_done
归属	target_component、assigned_role、assignee_id、created_by
调度	priority、queue_sequence、status、scheduled_at、timeout_seconds
上下文	target_repository、source_ref_constraints、repo_read_scopes、repo_write_scopes、attachment_refs
依赖	dependency_ids、blocking_reason、source_task_id
策略	max_retries、max_review_iterations、review_policy、task_completion_policy、queue_advance_policy、risk_level
交付	branch_name、commit_sha、merge_request_id、pipeline_id
追踪	active_run_id、audit_trace_id、created_at、updated_at

Run 建议字段

字段	说明
id / task_id / attempt_no	唯一运行及重试序号
requester / execution_principal / device_id	请求人、任务执行主体与设备
codex_thread_id / turn_id	官方 Codex 执行标识
review_thread_id	上下文隔离的自动审查标识
lease_generation / lease_expires_at	租约一致性

字段	说明
policy_hash / input_hash	权限和固定输入证明
base_refs / attachment_versions	各仓库与附件版本
result_commit / artifact_hash	执行产物
test_summary / error_class	测试和失败分类
execution_slot_released_at	Runner 执行槽释放时间
started_at / finished_at	运行时间

接口设计

主要 API

方法	路径	用途	权限
POST	/v1/tasks	创建任务并提交验收标准、依赖和附件引用	项目成员
GET	/v1/tasks	按项目、角色、状态和筛选读取看板	项目成员
PATCH	/v1/tasks/{id}	编辑未领取任务、暂停、恢复或取消	创建人/负责人
POST	/v1/queues/{id}/claim	原子领取下一条可执行任务	Runner
POST	/v1/runs/{id}/heartbeat	续租并上报健康和进度	租约持有 Runner
POST	/v1/runs/{id}/events	批量上传结构化执行事件	租约持有 Runner
POST	/v1/runs/{id}/complete	提交执行、测试和审查结果	租约持有 Runner
POST	/v1/runs/{id}/operations/authorize	为 Commit/Push/MR 等副作用签发短期操作授权	有效租约 Runner
POST	/v1/tasks/{id}/review	批准、请求修改、重新审查或人工接管	审查人
POST	/v1/tasks/{id}/dependencies	创建/关联跨团队依赖	授权用户/策略
POST	/v1/attachments	创建附件上传会话	项目成员
POST	/v1/gitlab/webhooks	同步 MR、Pipeline、审批和合并状态	签名或 X-Gitlab-Token
GET	/v1/events/stream	SSE 实时任务和 Runner 状态	已登录用户
POST	/v1/admin/emergency-stop	项目或全局停止领取	管理员

幂等与并发约束

- 所有创建、完成、审批和外部副作用接口支持 Idempotency-Key。
- 状态更新使用乐观锁 version；Runner 更新额外校验 lease_generation/fencing token。

- claim 接口在数据库事务中选择并更新任务，禁止客户端先 GET 后自行领取。
- MR、Commit、附件和 Operation 使用稳定业务键，重复调用必须返回已存在结果。
- Webhooks 按事件 ID 去重，并允许乱序到达后重新计算派生状态；周期性 reconciliation 对账 GitLab 实际状态。

示例任务请求

```
{
  "taskKey": "FE-1024",
  "projectId": "smart-device",
  "targetComponent": "frontend",
  "title": "实现设备状态面板",
  "description": "根据附件页面稿和后端接口实现设备状态面板。",
  "acceptanceCriteria": [
    "展示在线、离线、告警三种状态",
    "覆盖 loading 和 error 状态",
    "前端单元测试通过"
  ],
  "sourceRefConstraints": {
    "frontend": "main",
    "backend": "release/device-v3",
    "hardware": "v2.8.1",
    "contracts": "main"
  },
  "repoReadScopes": ["repo:frontend:read", "repo:backend:read", "repo:hardware:read", "repo:contracts:read"],
  "repoWriteScopes": ["repo:frontend:write"],
  "attachmentScopes": ["attachment:project:read"],
  "attachmentRefs": [{"id": "screen-spec", "version": 3}],
  "taskCompletionPolicy": "AFTER_MERGE",
  "queueAdvancePolicy": "AFTER_TASK_COMPLETE",
  "maxReviewIterations": 2,
  "riskLevel": "normal"
}
```


安全架构

威胁模型

系统面对的主要威胁包括角色伪造、越权目录写入、提示注入、恶意附件、密钥泄漏、网络外传、保护分支误操作、重复执行、审计缺失和自动化成本失控。安全策略必须由服务器、Runner、沙箱、操作系统和 GitLab 多层共同实施。

威胁	控制措施	失败处理
角色/项目伪造	SSO、服务器 RBAC、签名令牌、禁止信任客户端 role	拒绝并审计
越权文件写入	只读挂载、sandbox、Diff Guard、GitLab 权限	停止任务、禁止 Push、P0/P1 告警
提示注入	任务/源码/附件视为数据，系统策略不可覆盖	隔离相关输入，转人工
凭证泄漏	短期令牌、凭证代理、日志脱敏、Secret Scan	吊销、轮换、暂停 Runner
网络外传	任务命令默认无网络、出口代理/防火墙/容器网络策略与审计	阻断并告警
重复副作用	租约、幂等 Operation、外部结果对账	返回既有结果或转恢复
危险自动操作	风险分级、动作哈希、人工批准	暂停在审查列
成本失控	单任务预算、团队配额、循环上限、熔断	停止执行并通知

身份与凭证

- 用户使用公司 OIDC/SSO；Runner 建议使用设备授权流程或受管设备注册。
- 访问令牌短期有效，刷新令牌保存在 Windows Credential Manager 或公司凭据代理。
- GitLab 的 Git 传输与 MR API 使用不同主体或受控 Broker，凭证按目标项目、分支和操作短期签发，不进入 Codex/测试子进程。
- 对象存储使用任务级短期下载凭证或预签名 URL，不向 Codex 暴露长期密钥。
- 敏感配置、.env、签名证书、生产凭证默认不进入任务工作区和只读上下文。
- 权限撤销必须快速传播到任务服务器和 Runner，必要时停止当前运行。

文件、命令与网络安全

- 使用真实路径归一化防止 ../、软链接、junction、大小写和 UNC 路径绕过。
- 命令执行默认在沙箱内；项目只允许配置白名单或风险分级命令。
- Runner 自行执行的测试、构建、Lint、依赖安装以及附件/文档解析器，也必须运行在与 Codex 等同或更严格的隔离环境中，不得继承 Runner 服务凭证或完整环境变量。
- 任务正文不得直接指定 danger-full-access 或新增任意写目录。

- 必须区分 Codex 控制面网络与任务命令网络：Runner 和本地 Codex 运行时按批准路径访问 OpenAI 服务，任务命令默认关闭公网网络。
- Codex sandbox 的 networkAccess 仅表达网络开关，不等同于域名级白名单；公司 Git、包代理、测试服务和批准域名必须由出口代理、防火墙或容器网络策略强制限制并审计。
- 禁止向外部地址上传源码、附件、日志和构建产物，除非有专门审批。
- 对最终 Diff、生成文件、二进制和大文件执行策略检查；异常内容转人工。

数据保护与审计

- 通信全程 TLS；服务端数据库、对象存储和备份按公司标准加密。
- 任务、附件、日志和源码访问按项目隔离；跨项目访问需要显式授权。
- 日志默认脱敏 Token、Cookie、密码、Authorization header 和匹配的密钥模式。
- 审计事件在业务层按逻辑追加写，普通主体不得更新或删除；只有同时采用哈希链、权限隔离、远端归档以及对象锁/WORM 等控制后，才能宣称具备防篡改能力。
- 每个 Run 记录模型/运行时版本、提示模板版本、策略哈希、仓库 Commit 和附件哈希。
- 建立数据分级、保留、删除、法律保全和用户访问审计策略。

可靠性与故障恢复

故障分类

类别	示例	默认策略
瞬时故障	网络超时、GitLab 429、对象存储短暂不可用	指数退避、断路器、保持现场
可恢复执行故障	Runner 重启、Codex 进程中断	对账 thread/worktree/operation 后恢复
确定性失败	测试持续失败、编译错误、MR 冲突	有限返工后转人工
权限/安全失败	越权写入、密钥命中、恶意附件	立即停止，不自动重试
需求失败	验收标准冲突、输入不足	NEEDS_ATTENTION，要求补充
成本/循环异常	重复修改、Token 超预算	熔断并通知

恢复规则

- Runner 启动后先调用 reconciliation API 获取自身未完成 Run，而不是立即领取新任务。
- 若租约仍有效且 Codex/工作区可恢复，继续原 Run；否则等待服务器决定新 attempt。
- 执行完成但响应丢失时，通过 Commit、MR、Operation 和 artifact_hash 对账，禁止重复副作用。

- GitLab Push 成功但 MR 创建响应丢失时，按 source_branch 查询已有 MR。
- GitLab Webhook 只用于加速状态同步，不作为唯一一致性来源；任务服务器必须周期性 reconciliation MR、Pipeline、审批和合并状态，并修复漏发、乱序或处理失败的事件。
- Webhook 鉴权按 GitLab 实例版本选择：支持签名 Webhook 时验证签名和防重放信息；旧版本至少验证预配置 secret token (X-Gitlab-Token)、来源策略和事件唯一标识。
- 任务取消后停止后续 Commit/Push，保留现场到策略期限，允许管理员导出补丁。
- 无法证明安全恢复时宁可转人工，不自动从不确定状态继续。

可观测性

日志、指标与 Trace

- 所有请求、任务、Run、Codex thread、GitLab 操作和附件使用统一 trace_id。
- 结构化日志包含主体、阶段、状态、耗时、错误分类和重试，但不包含明文密钥。
- 指标覆盖 API、数据库、消息、GitLab、对象存储、Codex、Runner 和业务状态。
- 任务事件时间线用于用户查看；详细日志用于平台排障，两者分级授权。
- 成本指标按项目、团队、用户、任务类型、模型和 Runner 版本聚合。

关键告警

级别	场景	响应
P0	确认越权写入、保护分支被非授权修改、大规模凭证泄漏	全局停止、吊销凭证、事故响应
P1	调度完全停止、数据库故障、任务状态大面积错误	值班介入，目标 60 分钟恢复
P2	GitLab/Codex 集成持续失败、任务堆积、成本异常	项目级熔断和排查
P3	单任务失败、附件解析失败、单 Runner 离线	通知负责人并按策略重试

非功能需求

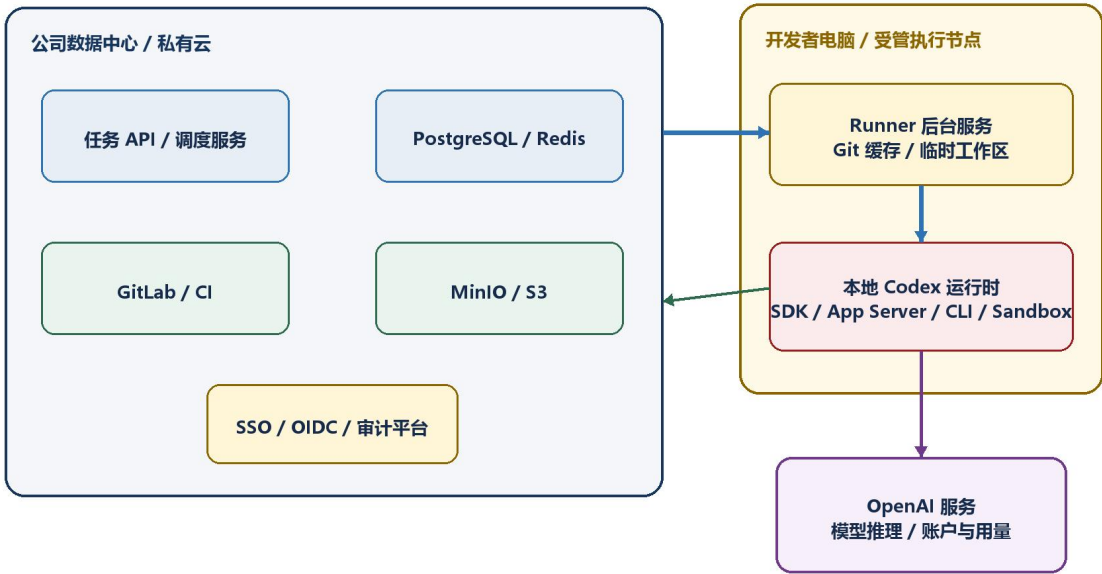
编号	指标	Production 目标
NFR-AVAIL-001	任务服务月度可用性	≥ 99.9%
NFR-API-001	任务查询 API 延迟	p95 ≤ 500 ms
NFR-API-002	普通状态写入延迟	p95 ≤ 800 ms
NFR-UI-001	初始看板加载	p95 ≤ 2.5 秒
NFR-EVENT-001	状态事件端到端可见延迟	p95 ≤ 5 秒

编号	指标	Production 目标
NFR-SCHED-001	空闲 Runner 发现新任务	p95 ≤ 30 秒
NFR-RUNNER-001	Runner 心跳 / 离线识别	≤ 30 秒 / ≤ 90 秒
NFR-SCHED-002	达到队列推进点到尝试领取下一项	p95 ≤ 30 秒
NFR-CONTROL-001	调度控制链路成功率	≥ 99.5%
NFR-CONSIST-001	任务与 GitLab 关联一致率	≥ 99.9%
NFR-IDEMP-001	非预期重复有效执行率	< 0.01%
NFR-SEC-001	非授权目录写入 / 保护分支直推	0 次
NFR-AUDIT-001	审计事件完整率	100%
NFR-DR-001	数据库 RPO / 核心服务 RTO	≤ 15 分钟 / ≤ 2 小时

度量说明 代码任务一次完成率不属于平台可用性 SLO。应单独跟踪首次审查通过率、平均返工次数、人工接管率和最终业务验收率。

部署架构

建议部署拓扑



本地任务需要执行节点和 Runner 在线；任务控制面、源码和附件由公司系统持久化，Codex 本地运行时再访问 OpenAI 服务。

图 4 公司控制面、Runner、本地 Codex 运行时与 OpenAI 服务的部署关系

公司侧服务

- 任务 API/调度服务：无状态部署，Production 至少两个实例。
- PostgreSQL：保存业务实体、状态机、租约、审计索引和配置。
- Redis/消息系统：缓存、事件广播、速率限制和可选队列，不作为唯一事实源。
- MinIO/S3：附件、测试报告、日志、构建产物和审查报告。
- GitLab/CI：源码与交付链路，可为现有公司实例。
- SSO/OIDC、监控、日志、告警和密钥管理复用公司基础设施。

开发者侧

- Runner 作为 Windows Service 或受管后台进程，支持开机启动、受控升级和一键停止。
- 本地保留 Git 对象缓存和短期工作区，配额、清理和加密遵循公司策略。
- Runner 只建立出站连接，避免要求公司服务器主动访问开发者电脑。
- 本地任务要求电脑和 Runner 在线；离线时服务器保留任务，不产生丢失。
- 集中执行场景可把 Runner 部署到受管工作站或容器节点，但权限和源码隔离规则不变。

实施策略与阶段计划

推荐路线 技术验证 → MVP → Pilot → Production。基线周期约 14–20 周，取决于现有 SSO、GitLab、对象存储和 CI/CD 的成熟度。

阶段	建议周期	核心目标	退出条件
阶段 0 技术验证	1–2 周	验证 Codex、工作区权限、GitLab、附件和恢复链路	至少 10 次真实端到端验证，无不可绕过阻塞
阶段 1 MVP	4–6 周	单项目、前端角色、单 Runner 串行自动执行	连续 30 个测试任务，无越权、无重复提交
阶段 2 Pilot	4–6 周	前端/后端/硬件、多仓库、跨团队依赖和 SSO	真实团队运行 4 周，平台链路成功率 ≥98%
阶段 3 Production	6–8 周	高可用、安全、成本、运维、评测和标准接入	连续 30 天满足 SLO，通过演练与安全评审

阶段 0：技术验证与架构冻结

- 验证 Runner 调用官方 Codex 完成真实仓库任务，并获得稳定 thread/run 标识。
- 验证流式事件、完成、取消、审批、恢复和 review/start 上下文隔离审查。
- 验证 frontend 可写、backend/hardware/contracts/attachments 只读的权限组合。
- 验证 Windows 下 worktree、只读挂载、sandbox 和 Diff Guard。
- 验证 GitLab Push、MR、CI Webhook、Code Owner 和保护分支。
- 验证附件上传、扫描、固定版本、下载、哈希和只读提供。
- 验证 claim、租约、心跳、Runner 崩溃、网络中断和幂等恢复。
- 确认后台 Codex thread、插件自定义 UI 与桌面原生导航的可见性边界，任务中心不得依赖未保证的永久侧栏注册能力。
- 交付物：端到端 PoC、CodexAdapter 契约、状态机、OpenAPI 初稿、权限模型和技术风险报告。
- 退出标准：覆盖成功、审查失败、Runner 中断、网络中断、GitLab 失败和越权尝试等不少于 10 次运行。

阶段 1：MVP

- 一个公司项目、一个前端队列、一个前端 GitLab 仓库可写。
- backend、hardware、contracts 作为固定 Commit 只读上下文。
- Windows 本地 Runner、插件/Web 四列看板、手机简化创建页。

- 单 Runner 串行执行，每任务独立 thread、分支和工作区。
- 隔离环境自动测试、上下文隔离的 Codex 审查、最多 2 次自动返工。
- GitLab MR 创建，人工最终合并；基础附件、审计和失败恢复。

MVP 工作包

工作包	核心内容
A 任务服务	身份、项目、任务、状态机、claim、租约、事件、审计和幂等 API
B Runner	设备登录、工作区、CodexAdapter、测试、Diff Guard、GitLab 和恢复
C 自动审查	上下文隔离审查、结构化结果、返工、确定性门禁和人工接管
D 任务 UI	四列看板、任务详情、日志、附件、MR、CI 和控制项
E 安全基础	RBAC、主体分离、沙箱、出口网络策略、Secret Scan、逻辑追加写审计

MVP 里程碑

里程碑	结果
M1	手机/Web 创建任务后进入前端待办列
M2	Runner 原子领取任务并准备隔离工作区
M3	Codex 完成一次代码修改、测试和结果回传
M4	审查失败可返工，通过可创建 GitLab MR
M5	按 queue_advance_policy 自动新建 thread 并处理下一项；MVP 默认任务完成后推进
M6	完成安全、恢复和端到端验收，进入内部试用

阶段 2 : Pilot

- 扩展到前端、后端、硬件三个角色和多个 GitLab Group。
- 接入公司 SSO/OIDC、多角色切换、跨团队子任务和依赖等待。
- 支持多个 Runner 和队列并发上限，加入优先级、截止时间和维护窗口。
- 接入企业 IM/邮件/推送、管理员后台、成本和团队配额。
- CI 通过后才进入最终完成；人工审查和 Codex 审查组合。
- 建议试点 1 个大项目、每角色 3-8 人、运行 4 周、累计至少 200 个任务。

Pilot 退出标准

- 无 P0/P1 安全事件，无非授权写入和保护分支直推。
- 平台调度链路成功率 $\geq 98\%$ ，状态与 GitLab 一致率 $\geq 99.5\%$ 。

- 重复有效执行率 <0.1%，且没有重复合并。
- 至少 30 个附件任务、20 个跨团队依赖任务和多类故障演练。
- 试点用户至少 70% 认可平台减少重复性工作。

阶段 3 : Production

- 任务服务高可用、数据库备份恢复、对象存储生命周期与灾难恢复。
- Runner 灰度发布、版本协商、一键回滚和 Codex 版本契约测试。
- 团队并发、Token/费用预算、异常熔断和全局暂停。
- 生产级监控、值班、事故手册、权限复核、Token 轮换和漏洞扫描。
- 版本化 Codex 质量评测集，任何 SDK/模型/提示升级前必须回归。
- 标准项目接入模板，以及硬件仿真、设备池或实验室审批流程。

团队分工

角色	主要职责
产品负责人	需求范围、流程、优先级、业务验收和推广
技术负责人	总体架构、状态机、权限模型和关键决策
平台后端	任务服务、调度、身份、审计、通知和管理 API
Runner/Codex 工程师	CodexAdapter、工作区、事件、恢复和审查编排
前端工程师	看板、详情、管理后台和实时状态
移动端工程师	任务创建、附件上传、状态和通知
DevOps/GitLab	仓库权限、CI、部署、监控和备份
安全工程师	威胁建模、密钥、沙箱、内容安全和渗透测试
QA	测试计划、评测集、故障演练和验收
领域代表	前端/后端/硬件规则、真实任务和验收标准

测试策略

单元与组件测试

- 状态机合法/非法迁移、队列排序、优先级和依赖判断。
- claim、租约、续租、过期、回收、版本锁和幂等键。
- 权限交集、目录白名单、真实路径和跨仓库写入拦截。
- Codex 事件解析、错误分类、取消、审批和恢复。
- 分支、Commit、MR、Webhook 鉴权/去重/乱序处理、周期性 reconciliation 和 CI 状态映射。
- 附件版本、哈希、扫描、日志脱敏、重试和熔断。
- 核心调度与权限模块单元测试覆盖率目标 $\geq 90\%$ ，整体后端 $\geq 80\%$ 。

集成与端到端测试

55. 手机创建前端任务并上传附件。
 56. 任务进入待办，Runner 自动领取。
 57. 准备 frontend 可写、其他仓库和附件只读的工作区。
 58. Codex 修改代码并执行必需测试。
 59. 上下文隔离的自动审查及确定性门禁通过，Runner 复核租约后推送分支并创建 MR。
 60. GitLab CI 和审批状态回写，任务进入完成。
 61. Runner 自动创建下一任务的新 thread 并继续执行。
- 覆盖审查不通过、自动返工超限、暂停、取消和人工接管。
 - 覆盖 Runner 崩溃、电脑重启、网络中断、GitLab/对象存储/Codex 不可用。
 - 覆盖 CI 失败、MR 冲突、依赖未满足、重复提交和两个 Runner 争抢。

安全测试

- 前端身份尝试写 backend/hardware、通过软链接和路径穿越绕过。
- 任务或附件包含“忽略系统规则”等提示注入内容。
- Git 子模块、脚本或子进程访问工作区外目录。
- 日志和产物泄漏 Token、Cookie、密码和 API Key。
- 伪造用户、角色、项目、Runner、租约 generation 或状态。
- 恶意/超大附件、无效 MIME、扫描失败和解析炸弹。
- Codex 直接推保护分支、访问公网、生产系统或未授权服务。

Codex 质量评测

模型结果具有非确定性，必须建立版本化评测集，固定源码 Commit、附件版本、验收标准、允许目录、必需测试和人工评分规则。每次 Codex 模型、SDK、App Server、提示模板、AGENTS 或 Runner 版本升级前都执行回归。

- 小型缺陷修复、页面功能、API 适配和单元测试补全。
- 前后端契约变更识别、硬件协议读取和跨团队依赖。
- 带 PDF、截图、日志、错误需求和不完整需求的任务。
- 越权修改、高风险命令、提示注入和成本循环诱导。

功能验收标准

身份与权限

- 用户通过公司认可的身份系统登录；UI 显示项目、角色和读写范围。
- 客户端修改 role 或 write_scopes 不能绕过服务器权限。
- 前端 Codex 能读取授权的多仓库快照，但不能修改或推送 backend/hardware。
- 所有权限拒绝记录用户、任务、Runner、时间、路径和原因。

任务看板

- 四列正常显示，正常网络下状态变化 5 秒内可见。
- 同一任务同一时刻只属于一个主业务列。
- 手机创建任务 10 秒内出现在看板；异常状态可辨识。
- 详情可查看 thread、Commit、MR、测试、附件、审查和审计。

自动执行

- 空闲 Runner 领取符合项目和角色的第一条可执行任务。
- 同一任务不能被两个 Runner 同时有效执行。
- 达到 queue_advance_policy 推进点后 30 秒内尝试领取下一项；每项任务使用独立 thread 和工作区。
- Runner 重启后能恢复、对账或安全创建新 attempt。

GitLab 与交付

- 任务记录所有仓库固定 SHA，执行中外部更新不改变输入。

- 目标仓库只在任务分支修改，保护分支不可直接 Push。
- 最终 Diff 经过授权检查，越权时不 Commit/Push。
- MR 关联任务、执行、测试和审查；完成条件遵循项目门禁。

执行与审查

- 执行日志可查看，必需测试被真实执行。
- 自动审查使用与执行隔离的上下文并结构化输出通过、修改或人工处理；不能替代 CI、Diff Guard 和人工门禁。
- 自动返工达到上限后停止循环。
- 执行成功、隔离测试通过、自动审查和确定性门禁通过、MR 创建构成自动交付就绪；最终完成还遵循 CI、审批和 task_completion_policy。

附件与审计

- 附件具有唯一 ID、版本、哈希、扫描状态和访问范围。
- Runner 只下载明确引用附件，执行期间版本固定且只读。
- 每项任务可追踪身份、策略、thread、仓库 SHA、附件、命令、测试、MR 和审批。

上线门禁

- 架构、安全、运维和数据合规评审通过。
- MVP 与 Pilot 验收标准满足，无未关闭 P0/P1 缺陷。
- 权限越权、调度恢复、备份恢复、Runner 回滚和故障演练通过。
- Codex 回归评测达到基线，GitLab 保护分支、Code Owner 和 CI 门禁生效。
- 费用预算、限额、熔断、全局暂停、告警和值班流程生效。
- 产品、技术、安全、运维和试点团队代表共同签署上线确认。

主要风险与缓解

风险	影响	缓解措施
Codex SDK/App Server 变化	Runner 兼容性中断	适配层、固定版本、契约测试、灰度升级
后台 thread 不显示在桌面侧栏	用户无法从原生入口查看	任务中心作为主入口，保存 thread_id 和摘要
模型结果非确定	质量和耗时波动	固定输入、评测集、上下文隔离审查、确定性检查和人工门禁
需求不完整	反复修改或错误实现	任务模板、必填验收、缺失信息转人工

风险	影响	缓解措施
越权修改	安全和协作事故	只读挂载、sandbox、Diff Guard、最小 Git 权限
附件提示注入	绕过规则或恶意命令	输入视为数据、网络限制、危险动作审批
Runner 掉电	任务卡死或重复	租约、心跳、恢复点、幂等副作用
GitLab/网络不可用	无法准备或交付	退避、断路器、保留现场、可恢复状态
跨团队依赖死锁	任务长期阻塞	循环检测、超时升级、人工解除
自动返工循环	成本失控	最大次数、时间/Token 预算、熔断
硬件任务依赖实物	无法完全自动验收	仿真优先、设备池、实验室审批
用户过度信任自动审查	缺陷进入主分支	保留 Code Owner、CI 和高风险人工审批

上线前待决策事项

产品与流程

- 任务最终完成是在 MR 创建、CI 通过、可合并还是实际合并。
- queue_advance_policy 是否保持 MVP 严格串行，或在 Pilot 改为自动交付就绪后释放执行槽。
- 自动返工次数、单任务超时、队列排序和抢占规则。
- 跨团队子任务是否自动创建；建议 Pilot 先人工确认。
- 手机首期是否支持审批、取消和暂停执行。

Codex 与 Runner

- MVP 以 SDK 还是 App Server 为主，以及统一 CodexAdapter 的最小能力集。
- Runner 运行在开发者电脑、受管工作站还是集中执行节点。
- Codex/OpenAI 凭证采用用户授权还是公司受管账户，以及费用和用量归属；该选择不得改变任务服务器的项目/角色授权。
- 模型、推理参数、提示模板和 AGENTS 是否项目级统一管理。

仓库与权限

- 最终确认多仓库或 monorepo；本方案默认多仓库。
- 各角色可读/可写仓库、contracts 修改责任和高风险文件目录。
- Git 传输主体、MR API 主体或 GitLab Broker 的粒度、短期授权方式、分支/Commit/MR 命名和 Code Owner 规则。

- Git LFS 与对象存储边界、工作区/分支/日志保留周期。

安全与运维

- 源码与任务内容是否满足公司使用所选 Codex 服务的合规要求。
- SSO、密钥管理、病毒扫描、敏感信息检测和出口网络控制方案。
- 预计用户、项目、并发、费用预算、SLO、RPO/RTO 和支持时间。
- Runner 自动升级机制、维护窗口和紧急暂停责任人。

附录 A：状态字典

状态	业务列	说明
QUEUED	待办	已入队，尚未满足全部执行条件
READY	待办	可被 Runner 领取
BLOCKED_DEPENDENCY	待办	等待前置任务
RETRY_WAIT	待办	等待退避后重试
PAUSED	待办	被用户或管理员暂停
SCHEDULED	待办	等待计划执行时间
CLAIMED	执行	已原子领取并持有租约
PREPARING	执行	准备仓库、附件和策略
RUNNING	执行	Codex 正在执行
RECOVERING	执行	中断后的恢复与对账
WAITING_TOOL_APPROVAL	执行	等待执行阶段的有限工具动作批准
WAITING_EXTERNAL_RESULT	执行	等待执行阶段的受控外部结果
AUTO_REVIEWING	审查	上下文隔离的 Codex 自动审查
CI_RUNNING	审查	等待 GitLab CI
CHANGES_REQUESTED	审查	需要返工
HUMAN_REVIEW	审查	等待人工审查
WAITING_MERGE_APPROVAL	审查	等待合并、发布或业务验收审批
READY_TO_MERGE	审查	满足项目合并条件
NEEDS_ATTENTION	审查	自动流程无法继续
SUCCEEDED	完成	达到项目完成条件
MERGED	完成	MR 已合并
CANCELED	完成	被用户或管理员取消
REJECTED	完成	人工终止
SUPERSEDED	完成	被其他任务替代

附录 B：参考资料与能力依据

R1. OpenAI Codex Plugins — <https://developers.openai.com/codex/plugins>。插件、Skills、MCP 和可选 UI 的官方说明。

R2. OpenAI Codex SDK — <https://developers.openai.com/codex/sdk>。自动化任务、thread 启动/恢复和 SDK 定位。

R3. OpenAI Codex App Server — <https://developers.openai.com/codex/app-server>。thread、turn、事件、review 和 sandbox 协议。

R4. OpenAI Codex Security — <https://developers.openai.com/codex/security>。沙箱、审批和网络安全边界。

R5. OpenAI AGENTS.md — <https://developers.openai.com/codex/guides/agents-md>。项目级和目录级持久指导。

R6. OpenAI Git Worktrees — <https://developers.openai.com/codex/app/worktrees>。独立工作区与任务隔离。

R7. GitLab Protected Branches — <https://docs.gitlab.com/user/project/repository/branches/protected/>。保护分支和 Push/Merge 权限。

R8. GitLab Code Owners — <https://docs.gitlab.com/user/project/codeowners/>。目录责任人和审批规则。

R9. GitLab Large Files / Git LFS — https://docs.gitlab.com/topics/git/file_management/。大型二进制文件管理。

以上外部能力依据按 2026 年 8 月 7 日可获取的官方文档整理。Codex 和 GitLab 能力可能迭代，正式开发和升级前应重新执行兼容性验证。

附录 C：需求基线签署建议

评审角色	姓名/签字	日期	结论
产品负责人			通过 / 有条件通过 / 退回
技术负责人			通过 / 有条件通过 / 退回
安全负责人			通过 / 有条件通过 / 退回
DevOps/GitLab 负责人			通过 / 有条件通过 / 退回
前端代表			通过 / 有条件通过 / 退回
后端代表			通过 / 有条件通过 / 退回
硬件代表			通过 / 有条件通过 / 退回

文档结束 本说明书形成当前需求与架构基线。进入开发前，应先完成阶段 0 技术验证，并根据验证结果更新接口契约、周期、范围和风险。